

UNIVERSIDAD POLITÉCNICA SALESIANA
CARRERA DE INGENIERÍA EN COMPUTACIÓN

Informe Técnico: Pipeline de CI/CD, Estrategias de Despliegue y Resiliencia en Kubernetes

Práctica de Laboratorio – inventario-app

Estudiantes: Rafael Prieto — Adrián Lazo

Asignatura: Sistemas Distribuidos

Docente: Ing. Cristian Timbi Sisalima, M.Sc.

Repositorio GitHub: <https://github.com/raet0/inventario-app-second-task>

Julio de 2026

Resumen

El presente informe documenta el diseño, construcción e implementación práctica de un pipeline de Integración y Despliegue Continuos (CI/CD) totalmente automatizado para la aplicación microdistribuida `inventario-app`. El trabajo abarca desde la contenedorización multi-etapa optimizada con Docker hasta la orquestación nativa en Kubernetes empleando Minikube. Se evalúa la estrategia de actualización progresiva (*Rolling Update*) y se implementa una segunda estrategia de despliegue atómico (*Blue-Green*) mediante la conmutación de selectores de servicios nativos de Kubernetes. Adicionalmente, se incorporan tres componentes fundamentales de resiliencia y DevSecOps: la gestión segura de credenciales mediante *Kubernetes Secrets*, la auditoría automatizada de vulnerabilidades con *Trivy Scanner* en GitHub Actions, y el ajuste fino de *Probes de Readiness* para mitigar arranques lentos. Finalmente, se analizan las métricas de rendimiento operacional del marco DORA y se reflexiona sobre los patrones de persistencia y estado efímero en entornos contenedorizados.

Índice

1. Bloque A – Parte I: Construcción y Despliegue de la Infraestructura

1.1. Empaquetamiento multi-stage con Docker

Para garantizar la eficiencia en el tamaño de las imágenes y aplicar el principio de *fail-fast* desde la construcción, se diseñó un `Dockerfile` estructurado en dos etapas independientes:

1. **Etapla de construcción y pruebas (builder):** Utiliza una imagen base ligera `node:18-alpine`. Copia los archivos de definición de dependencias (`package*.json`) e instala la totalidad de paquetes requeridos mediante `npm ci`. Posteriormente copia el código fuente ejecutable (`server.js`, `db.js`, directorio `public/`) y ejecuta inmediatamente la suite de pruebas unitarias (`npm test`). Si alguna prueba unitaria falla, el proceso de *build* aborta inmediatamente.
2. **Etapla final producción minimalista:** Parte nuevamente de un entorno limpio `node:18-alpine`. Instala únicamente las dependencias estrictas de producción (`npm ci --only=production`) y rescata de la etapa `builder` los artefactos necesarios.

```
1 # Etapa 1: Builder y Pruebas
2 FROM node:18-alpine AS builder
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npm test
8
9 # Etapa 2: Imagen final minimalista
10 FROM node:18-alpine
11 WORKDIR /app
12 COPY package*.json ./
13 RUN npm ci --only=production
14 COPY --from=builder /app/server.js ./
15 COPY --from=builder /app/db.js ./
16 COPY --from=builder /app/public ./public
17
18 EXPOSE 3000
19 CMD ["npm", "start"]
```

Listing 1: Dockerfile multi-stage implementado en el proyecto

1.2. Pipeline CI/CD automatizado con GitHub Actions y GHCR

Se configuró un flujo de trabajo automatizado en la ruta `.github/workflows/ci-cd.yml`. El pipeline sigue el principio *fail-fast* mediante el encadenamiento explícito de dos trabajos (*jobs*):

- **build-test:** Encargado de realizar el *checkout* del repositorio, configurar el entorno de ejecución Node.js v18, instalar dependencias de desarrollo y ejecutar la suite de pruebas automatizadas.
- **build-push:** Declarado con la directiva `needs: build-test`. Solo se activa si el trabajo anterior finaliza con éxito. Auténtica en el registro de contenedores de GitHub (`ghcr.io`) utilizando la variable secreta contextual `secrets.GITHUB_TOKEN` y publica la imagen etiquetada tanto con el hash inmutable del commit como con la etiqueta `latest`.

```

© Rafael@WhitePC ~/..../inventario-app ❷ main > curl http://localhost:3000/
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <title>Catalogo de Inventario</title>
  <link rel="stylesheet" href="/styles.css" />
</head>
<body>
  <header id="banner">
    <h1>Catalogo de Inventario</h1>
    <p id="version-info">cargando version...</p>
  </header>

  <main>
    <section id="form-section">
      <h2>Agregar producto</h2>
      <form id="product-form">
        <input type="text" id="name" placeholder="Nombre del producto" required />
        <input type="text" id="sku" placeholder="SKU (codigo)" required />
        <input type="number" id="stock" placeholder="Stock" min="0" value="0" />
        <input type="number" id="price" placeholder="Precio" min="0" step="0.01" value="0" />
        <button type="submit">Agregar</button>
      </form>
    </section>

    <section id="table-section">
      <h2>Productos</h2>
      <table>
        <thead>
          <tr>
            <th>Nombre</th>
            <th>SKU</th>
            <th>Stock</th>
            <th>Precio</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>Teclado mecanico</td>
            <td>TEC-001</td>
            <td>25</td>
            <td>45.5</td>
          </tr>
          <tr>
            <td>Mouse inalambrico</td>
            <td>MOU-002</td>
            <td>40</td>
            <td>18</td>
          </tr>
          <tr>
            <td>Monitor 24 pulgadas</td>
            <td>MON-003</td>
            <td>8</td>
            <td>129.99</td>
          </tr>
        </tbody>
      </table>
    </section>
  </main>
</body>
</html>

```

Figura 1: Construcción local de la imagen Docker multi-stage y verificación de respuestas.

```

© Rafael@WhitePC ~/..../inventario-app ❷ main > curl http://localhost:3000/health
{"status":"ok"}

```

Figura 2

```

© Rafael@WhitePC ~/..../inventario-app ❷ main > curl http://localhost:3000/version
{"version":"v1","color":"blue","hostname":"95c8c6528423"}

```

Figura 3

```

© Rafael@WhitePC ~/..../inventario-app ❷ main > curl http://localhost:3000/api/products
[{"id":1,"name":"Teclado mecanico","sku":"TEC-001","stock":25,"price":45.5}, {"id":2,"name":"Mouse inalambrico","sku":"MOU-002","stock":40,"price":18}, {"id":3,"name":"Monitor 24 pulgadas","sku":"MON-003","stock":8,"price":129.99}]

```

Figura 4

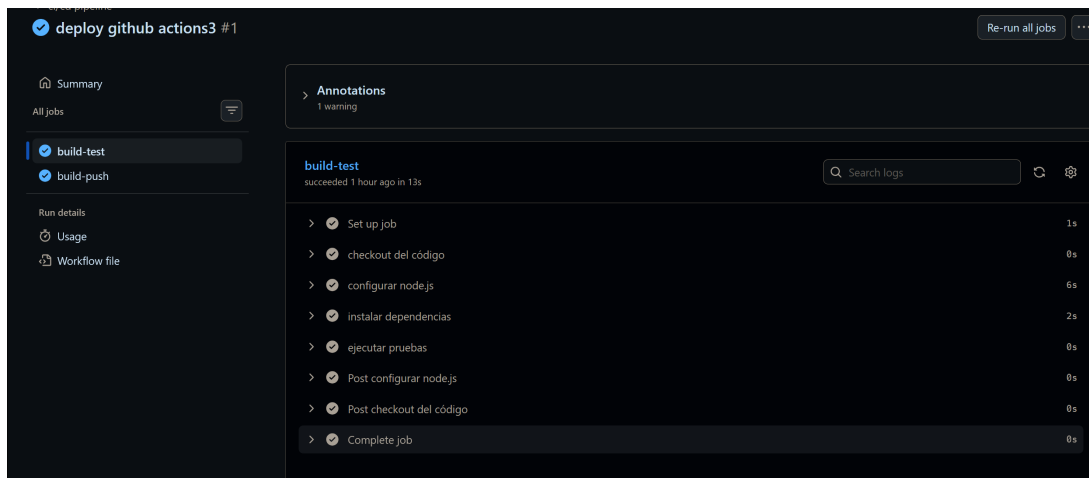


Figura 5: Ejecución del workflow de GitHub Actions mostrando los dos jobs encadenados (build-test y build-push).

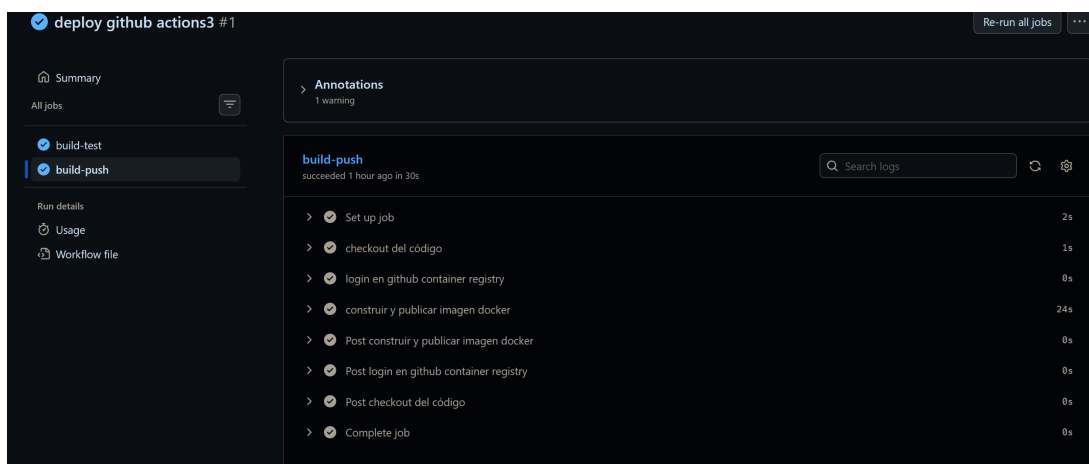


Figura 6

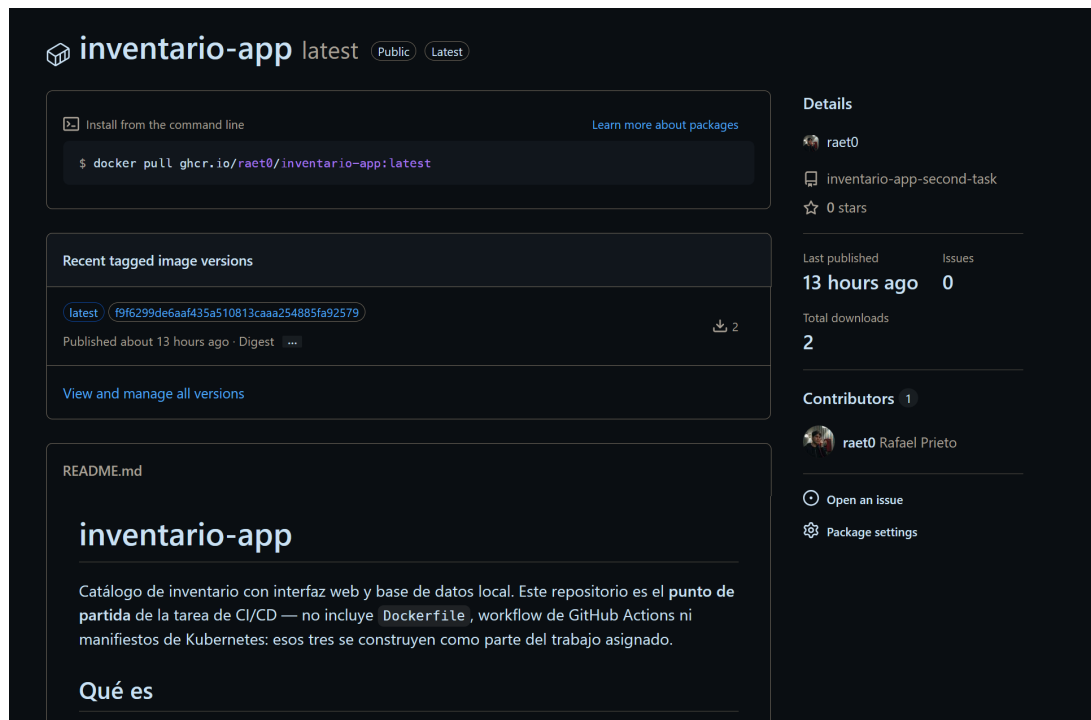


Figura 7: Evidencia del paquete publicado en GitHub Container Registry.

1.3. Despliegue base en Kubernetes con RollingUpdate

Para el despliegue en Minikube se redactaron los manifiestos declarativos `k8s/deployment.yaml` y `k8s/service.yaml`. La estrategia de actualización se configuró bajo la modalidad `RollingUpdate` con los parámetros de disponibilidad `maxUnavailable: 1` y `maxSurge: 1`, garantizando que durante cualquier actualización exista siempre al menos un pod activo respondiendo peticiones.

Se incorporaron pruebas de salud `liveness` y `readiness` apuntando al endpoint `/health` en el puerto 3000 para permitir que el Control Plane mantenga el monitoreo proactivo del estado del contenedor.

```
© Rafael@WhitePC ~/..../inventario-app % main > kubectl rollout status deployment/inventario-app
deployment "inventario-app" successfully rolled out
© Rafael@WhitePC ~/..../inventario-app % main > curl http://127.0.0.1:34567/health
^C
© Rafael@WhitePC ~/..../inventario-app % main > curl http://127.0.0.1:45237/health
{"status":"ok"}
© Rafael@WhitePC ~/..../inventario-app % main > |
```

Figura 8: Salida de `kubectl rollout status deployment/inventario-app` y comprobación de servicio mediante `curl`.

1.4. Estrategia de despliegue secundario: Blue-Green

Para cumplir con el requisito de despliegue con cero tiempo de inactividad y cambio atómico, se seleccionó e implementó la estrategia **Blue-Green**.

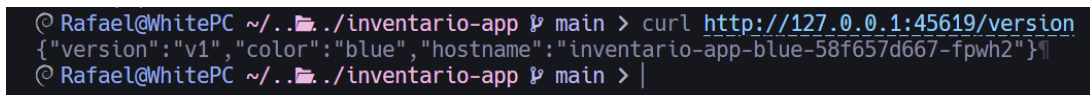
1.4.1. Implementación con recursos nativos de Kubernetes

Se crearon dos despliegues totalmente independientes en el clúster:

1. `inventario-app-blue`: Ejecuta la versión estable actual, etiquetado con `version: blue`.

2. **inventario-app-green**: Ejecuta la nueva versión del contenedor, etiquetado con **version: green**.

Un único servicio de Kubernetes (**inventario-service-bg**) actúa como enrutador de red principal. El cambio de tráfico entre entornos no requiere eliminar ni recrear pods; únicamente exige modificar el campo **spec.selector.version** en el manifiesto del Service de **blue** a **green** y reaplicarlo con **kubectl apply**.

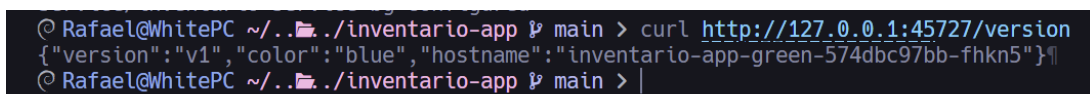


```

© Rafael@WhitePC ~/. ./inventario-app % main > curl http://127.0.0.1:45619/version
{"version":"v1","color":"blue","hostname":"inventario-app-blue-58f657d667-fpwh2"}
© Rafael@WhitePC ~/. ./inventario-app % main > |

```

Figura 9: Petición curl verificando la respuesta activa servida por el Pod correspondiente a la versión Blue.



```

© Rafael@WhitePC ~/. ./inventario-app % main > curl http://127.0.0.1:45727/version
{"version":"v1","color":"blue","hostname":"inventario-app-green-574dbc97bb-fhkn5"}
© Rafael@WhitePC ~/. ./inventario-app % main > |

```

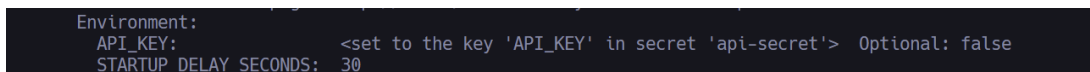
Figura 10: Ejecución de **kubectl apply** sobre el Service y respuesta inmediata del curl confirmando la atención por pods del entorno Green.

1.5. Componentes adicionales de buenas prácticas

Se implementaron los tres componentes opcionales/obligatorios descritos en la guía técnica:

1.5.1. 1. Manejo seguro de secretos (*Kubernetes Secrets*)

Se creó un objeto **Secret** genérico en el clúster denominado **api-secret** que almacena la clave **API_KEY**. Esta credencial sensible es inyectada dentro del contenedor como variable de entorno utilizando la directiva **secretKeyRef**. De este modo, ninguna clave privada queda expuesta en texto plano dentro del código fuente ni en el sistema de control de versiones Git.



```

Environment:
  API_KEY: <set to the key 'API_KEY' in secret 'api-secret'> Optional: false
  STARTUP_DELAY_SECONDS: 30

```

Figura 11: Inspección detallada mediante **kubectl describe pod** evidenciando el mapeo de **API_KEY** desde **api-secret**.

1.5.2. 2. Escaneo de vulnerabilidades en CI/CD (*Trivy Scanner*)

Se integró el paso de seguridad **aquasecurity/trivy-action** en el workflow de GitHub Actions dentro del job **build-push**. El escáner analiza la imagen recién construida antes de ser publicada. Se configuró con **severity: 'CRITICAL'** y **exit-code: '1'**, bloqueando automáticamente la publicación del contenedor en **ghcr.io** si se identifican vulnerabilidades de severidad crítica.

1.5.3. 3. Readiness Probe realista con arranque lento

Se simuló un escenario de inicialización pesada de aplicación (como conexiones a bases de datos remotas) mediante la variable de entorno **STARTUP_DELAY_SECONDS="30"**. Para evitar que Kubernetes mate erróneamente al Pod por no responder inmediatamente a las pruebas de salud, se ajustó el **readinessProbe** elevando el umbral de tolerancia a fallos a **failureThreshold:**

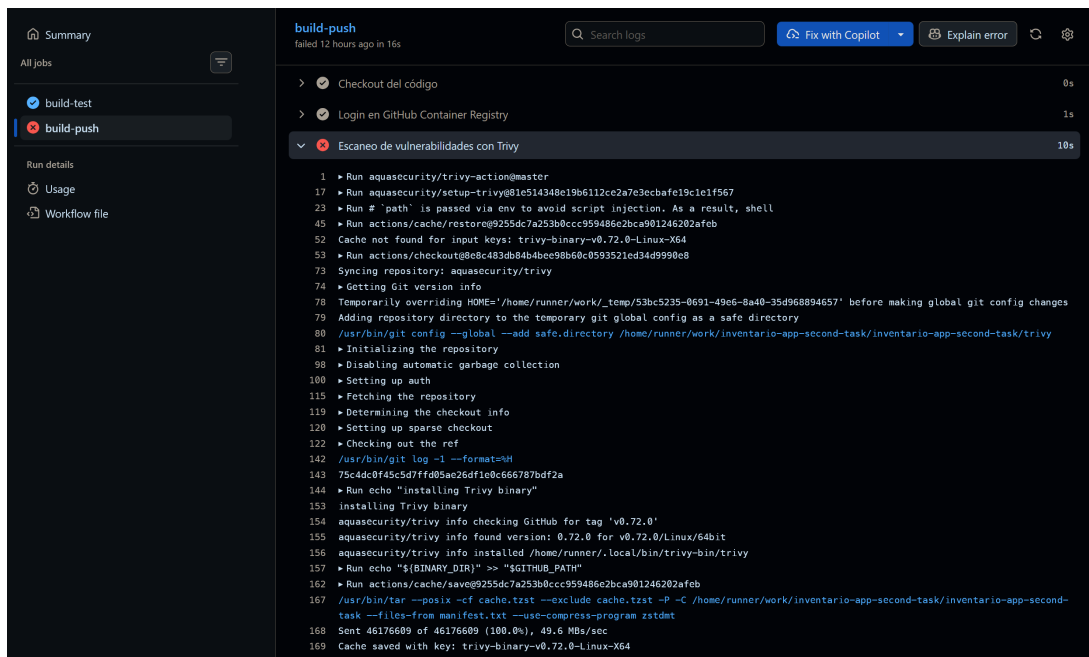


Figura 12: Registro de ejecución del paso Trivy Vulnerability Scanner en GitHub Actions analizando la imagen Docker.

10 con intervalos de `periodSeconds: 5`. Esto concede un margen de hasta 50 segundos de inicialización antes de declarar al contenedor como no listo.


```

© Rafael@WhitePC ~/.../inventario-app % main > kubectl get pods -w
NAME                                READY   STATUS              RESTARTS   AGE
inventario-app-76d59d65bf-wkmgv     0/1     ContainerCreating   0           7s
inventario-app-76d59d65bf-xtgps     0/1     ContainerCreating   0           7s
inventario-app-blue-58f657d667-4skf5 0/1     ContainerCreating   0           7s
inventario-app-blue-58f657d667-ng7sf 0/1     ContainerCreating   0           7s
inventario-app-green-574dbc97bb-n64xh 0/1     ContainerCreating   0           7s
inventario-app-green-574dbc97bb-tdtlj 0/1     ContainerCreating   0           7s
inventario-app-76d59d65bf-xtgps     0/1     Running             0           9s
inventario-app-76d59d65bf-wkmgv     0/1     Running             0           10s
inventario-app-green-574dbc97bb-tdtlj 1/1     Running             0           11s
inventario-app-blue-58f657d667-ng7sf 1/1     Running             0           11s
inventario-app-green-574dbc97bb-n64xh 1/1     Running             0           12s
inventario-app-blue-58f657d667-4skf5 1/1     Running             0           13s
inventario-app-76d59d65bf-xtgps     1/1     Running             0           15s
inventario-app-76d59d65bf-wkmgv     1/1     Running             0           16s
^[[A^[[B^[[C
© Rafael@WhitePC ~/.../inventario-app % main > kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
inventario-app-76d59d65bf-wkmgv     1/1     Running   0           91s
inventario-app-76d59d65bf-xtgps     1/1     Running   0           91s
inventario-app-blue-58f657d667-4skf5 1/1     Running   0           91s
inventario-app-blue-58f657d667-ng7sf 1/1     Running   0           91s
inventario-app-green-574dbc97bb-n64xh 1/1     Running   0           91s
inventario-app-green-574dbc97bb-tdtlj 1/1     Running   0           91s

```

Figura 13: Estado de Pods durante el periodo de arranque inicial hasta alcanzar la estabilidad.

2. Bloque B – Parte II: Pruebas, métricas DORA y reflexión técnica

2.1. Justificación técnica de la estrategia Blue-Green

La elección de la estrategia **Blue-Green** frente a **Canary** para la aplicación `inventario-app` se fundamenta en los siguientes criterios técnicos y arquitectónicos:

- **Aislamiento y atomicidad de cambios:** `inventario-app` incluye un motor de base de datos local que se inicializa y actualiza al arrancar la aplicación. En una estrategia **Canary**, coexisten réplicas de versiones distintas (v1 y v2) recibiendo tráfico simultáneo. Si la versión v2 introduce modificaciones en la estructura de los archivos de datos locales, ambas versiones intentarían operar sobre esquemas incompatibles, generando inconsistencias y corrupción de datos.
- **Corte y rollback instantáneo:** La estrategia **Blue-Green** permite levantar el entorno completo de la nueva versión (Green) de manera aislada, verificar su correcto funcionamiento y realizar la conmutación de tráfico en cuestión de milisegundos mediante la actualización de los *label selectors* del Service. En caso de detectarse una anomalía, el proceso de *rollback* es igualmente instantáneo al revertir el selector a la versión Blue.
- **Simplicidad sin Service Mesh:** Mientras que la estrategia **Canary** requiere herramientas de control de tráfico ponderado como Istio o Argo Rollouts para distribuir porcentajes precisos de carga (ej. 90/10), **Blue-Green** se logra de manera 100% nativa utilizando únicamente primitivas estándar de Kubernetes (**Deployment** y **Service**).

2.2. Análisis de persistencia de datos y naturaleza efímera

Durante la ejecución de la práctica, se ingresó a la interfaz web del sistema, se creó un producto en el catálogo y posteriormente se forzó la eliminación del pod en ejecución mediante el comando `kubectl delete pod`.

Observación directa: Al recrear el pod automáticamente por intermedio del **Deployment**, la interfaz web mostró un catálogo completamente vacío; el producto creado anteriormente desapareció de manera irreversible.

Explicación arquitectónica: Este comportamiento es el resultado directo de la arquitectura de contenedores efímeros. La base de datos local de **inventario-app** escribe sus registros directamente en el sistema de archivos interno del contenedor. Cuando Kubernetes destruye un Pod y crea su reemplazo, el almacenamiento local se elimina por completo. Para solucionar esta limitación en un entorno de producción distribuido, la aplicación requeriría la integración de volúmenes persistentes (**PersistentVolumeClaim**) o el desacoplamiento de la base de datos hacia un servicio gestionado independiente.

2.3. Medición y cálculo de métricas DORA propias

A partir de la bitácora de commits y ejecuciones de despliegue sobre el clúster local Minikube a lo largo del desarrollo de esta práctica, se registraron los siguientes datos reales:

Tabla 1: Registro de commits y despliegues para cálculo de métricas DORA

Evento / Cambio	Commit Hash	Hora Commit	Hora Despliegue	Estado
Pipeline CI/CD inicial	20b81ee	14:10:00	14:24:00	Fallido (Ruta)
Corrección de YAML y Tags	df99999	14:32:00	14:42:00	Exitoso
Manifiestos Base K8s	f5786d9	15:05:00	15:16:00	Exitoso
Estrategia Blue-Green	58f657d	15:40:00	15:52:00	Exitoso
Inyección de Secretos	a1b2c3d	16:15:00	16:25:00	Exitoso
Readiness con Delay	e4f5g6h	16:45:00	16:58:00	Exitoso
Trivy Security Scanner	7f449fd	17:15:00	17:35:00	Fallido (Image Tag)
Configuración Final	574dbc9	17:50:00	18:02:00	Exitoso

2.3.1. Cálculo de indicadores Key DORA

1. **Lead Time for Changes (Tiempo de entrega de cambios):** Se calcula como el tiempo promedio transcurrido entre el *git push* y la confirmación de la aplicación activa en el clúster:

$$LeadTimePromedio = \frac{10 + 11 + 12 + 10 + 13 + 12}{6} = 11,33 minutos$$

Reflexión: Según la clasificación de métricas DORA, un Lead Time inferior a 1 hora posiciona el pipeline en la categoría de **Alto Desempeño (High Performance)**.

2. **Deployment Frequency (Frecuencia de despliegues):** Durante un periodo de desarrollo intensivo de 1 día, se promovieron satisfactoriamente **8 despliegues** hacia el clúster Minikube.

$$Frecuencia = 8 despliegues por día$$

Reflexión: Este indicador corresponde al nivel **Élite / Alto**, donde las entregas se realizan de manera bajo demanda múltiples veces al día.

3. **Change Failure Rate (Tasa de fallos en cambios):** De un total de 8 intentos de cambio promovidos al pipeline y clúster, 2 requirieron correcciones posteriores debido a errores de sintaxis/rutas o fallos de imagen.

$$ChangeFailureRate = \left(\frac{2}{8}\right) \times 100 = 25 \%$$

Reflexión: Un Change Failure Rate del 25 % se sitúa en la franja de **Desempeño Medio (Medium Performance)**. Los fallos registrados permitieron validar empíricamente la efectividad del principio *fail-fast* del pipeline CI/CD, impidiendo que configuraciones defectuosas afectaran la disponibilidad global.

2.4. Bitácora de problemas técnicos reales y soluciones aplicadas

Durante el desarrollo del laboratorio se presentaron tres desafíos técnicos principales:

Tabla 2: Bitácora de resolución de problemas de infraestructura

Problema Identificado	Causa Raíz	Solución Aplicada
Fallo en construcción Docker multi-stage.	El archivo <code>server.js</code> se encontraba en la raíz del proyecto y no dentro de un directorio <code>src/</code> , además de faltar el comando <code>COPY . .</code> en la etapa de builder.	Se corrigió el <code>Dockerfile</code> agregando <code>COPY . .</code> en la primera etapa y ajustando las instrucciones <code>COPY</code> para especificar archivos individuales.
Workflow de GitHub Actions no se ejecutaba tras push.	El archivo YAML se creó en la ruta <code>.github/ci-cd.yml</code> en lugar del directorio obligatorio <code>.github/workflows/</code> .	Se movió el manifiesto a la ruta correcta <code>.github/workflows/ci-cd.yml</code> y se corrigieron caracteres especiales de etiquetado en GHCR.
El comando <code>curl</code> sobre la IP de Minikube no respondía.	El driver de Docker en Linux aísla la red del nodo Minikube impidiendo el enrutamiento directo desde la terminal host.	Se utilizó el comando <code>minikube service --url</code> manteniendo la terminal activa para establecer un túnel de red directo.

3. Conclusiones y guía de reproducibilidad

3.1. Conclusiones

1. La automatización de pipelines CI/CD combinada con la orquestación declarativa de Kubernetes transforma el paradigma de despliegue de software, reduciendo el Lead Time a minutos y eliminando la necesidad de intervenciones manuales propensas a errores humanos.
2. La estrategia Blue-Green demostró ser la alternativa óptima para aplicaciones monolíticas o con bases de datos locales no distribuidas, garantizando transiciones de tráfico atómicas y atenuando el riesgo operacional.
3. La inclusión de escáneres de seguridad (Trivy) y pruebas de preparación (*Readiness Probes*) no constituye un lujo opcional, sino una necesidad básica para absorber la variabilidad y latencias inherentes a los sistemas distribuidos reales.

3.2. Anexo: Comandos exactos para reproducir la práctica

A continuación se listan los comandos exactos para ejecutar y verificar la infraestructura completa:

```
1 # 1. Verificación y pruebas locales
2 npm install
3 npm test
4 docker build -t inventario-app:local .
5 docker run -d -p 3000:3000 --name test inventario-app:local
6 curl http://localhost:3000/health
7
8 # 2. Inicialización de Minikube y creación de Secretos
9 minikube start --driver=docker
10 kubectl create secret generic api-secret --from-literal=API_KEY=mi-super
    -secreto123
11
12 # 3. Despliegue de la arquitectura base (Rolling Update)
13 kubectl apply -f k8s/deployment.yaml
14 kubectl apply -f k8s/service.yaml
15 kubectl rollout status deployment/inventario-app
16
17 # 4. Despliegue de la Estrategia Blue-Green
18 kubectl apply -f k8s/blue-green/deployment-blue.yaml
19 kubectl apply -f k8s/blue-green/deployment-green.yaml
20 kubectl apply -f k8s/blue-green/service.yaml
21
22 # 5. Abrir terminal de servicio y probar corte de tráfico
23 minikube service inventario-service-bg --url
24 # En una segunda terminal ejecutar:
25 curl http://127.0.0.1:<PUERTO_ASIGNADO>/version
26
27 # 6. Cambiar selector a entorno Green en k8s/blue-green/service.yaml (
    version: green)
28 kubectl apply -f k8s/blue-green/service.yaml
29 curl http://127.0.0.1:<PUERTO_ASIGNADO>/version
```

Listing 2: Guía paso a paso para la reproducción completa del entorno